Experiment 5: Singly Linked List Operations.

Problem Statement:
DoCE has student's club named 'Pinnacle Club'. Students of SY, TY & LY of dept. can be granted membership on request. Similarly one may cancel the membership of club. First node is reserved for president of club & last for secretary of club. WACP to maintain club member's info using SLL. Store student PRN & Name. Write func:
a) Add & delete the members awa president or even secretary.
b) Complete total nos. of club members
c) Display members
d) Sorting of two LL
e) Merging of two LL
f) Reversing using the pointers.

Objective:
i. To understand the concept of a SLL & dynamic memory allocation, using pointers in C
ii. To perform insertion, deletion, display, count nodes, sort, reverse opsn. on a SLL.
iii. To develop a menu driven application for workshop enrollment management

Platform:
i. 64-bit Open source Linux or its derivatives
ii. Open source C Programming tool like Gcc / Eclipse Editor.

Test Conditions

Test Case	Input / Operation	Expected Output
TC1	Add the President	President's PRN & Name are stored successfully as the first node
TC2	Add the Secretary	Secretary's PRN & Name are stored successfully as the last node.
TC3	Add a regular club member	Student rec is added successfully to the LL
TC4	Add multiple members	All student rec are stored in the LL in specified order
TC5	Display an empty club list	Message "No students registered"
TC6	Display all club members	All students rec, including the President & secretary, are displayed correctly.
TC7	Dlt an existing reg. member	The selected student's rec is deleted successfully.
TC8	Dlt the President or Secretary	The selected spl member is removed successfully & the LL/head are updated correctly.
TC9	Dlt a non-existing PRN	Msg: "student Not Found"
TC10	Count club members	The correct total nos of registered club members is displayed.
TC11	Sort the list by PRN	Student rec are displayed in asc. order of PRN
TC12	Merge two LL	The rec from both lists are combined into one LL in the specified order
TC13	Reverse the LL using three pointers	The order of student rec is reversed correctly
TC14	Dlt all members & display	The list becomes empty & an appropriate msg, such as "No students registered" is displayed

Pseudo Codes:

```
Structure Node:
        int PRN
        string Name
        pointer next
End Structure

Algorithm addPresident(head, prn, name):
create new_node
        set new_node.PRN = prn
        set new_node.Name = name
        set new_node.next = head
        display "President added successfully"
        return head
end addPresident

Algorithm addSecretary(head, prn, name):
Create new_node
set new_node.PRN = prn
set new_node.Name = name
set new_node.next = NULL
if head == NULL:
        set head = new_node
        return head
set temp = head
while temp.next != NULL:
        temp = temp.next
set temp.next = new_node
Display "Secretary added successfully"
Return head
```

End addSecretary.

```
Algorithm addMember(head, prn, name, pos):
        Create new-node
        set new-node.PRN = prn
        set new-node.Name = name
        set temp = head
        for i = 1 to pos - 2:
            if temp != NULL:
                temp = temp.next
        if temp == NULL:
            display "invalid position"
            return head
        set new-node.next = temp.next
        set temp.next = new-node
        display "Member added successfully"
        return head
End addMember

Algorithm deletePresident(head):
        if head == NULL:
            display "Club list is empty"
            return NULL
        set temp = head
        set head = head.next
        free temp memory
        Display "President deleted successfully"
        return head
End deletePresident.
```

Algorithm deleteSecretary(head):

```
if head == NULL:
    display "Club list is empty"
    return NULL
if head.next == NULL:
    free head
    return NULL
set temp = head
while temp.next.next != NULL:
        temp = temp.next
free temp.next
set temp.next = NULL
display "Secretary deleted successfully"
return head
```

End deleteSecretary

Algorithm deleteMember(head, prn):

```
if head == NULL:
    display "Club list is empty"
    return NULL
if head.PRN = prn:
    return deletePresident(head)
set temp = head
while temp.next != NULL and temp.next.PRN != prn:
        temp = temp.next
if temp.next == NULL:
    display "Member PRN not found"
    return head
set del-node = temp.next
set temp.next = del-node.next
```

```
        free del-node
        display "Member deleted successfully"
        return head
End deleteMember

Algorithm displayMembers(head):
        if head == NULL:
            display "club is empty"
            return
        set temp = head
        display "Pinnacle Club Member List:"
        while temp != NULL:
            display "PRN:" + temp.PRN + ", Name: " + temp.Name
            temp = temp.next
End displayMembers

Algorithm countMembers(head):
        set count = 0
        set temp = head
        while temp != NULL:
            count = count + 1
            temp = temp.next
        return count
End countMembers
```

```
Algorithm reverseList(head):
        set prev = NULL
        set current = head
        set next_node = NULL
        while current != NULL:
            next_node = current.next
            current.next = prev
            prev = current
            current = next_node
        set head = prev
        display "List reversed successfully"
        return head
End reverseList
```

```
Algorithm sortList(head):
        if head == NULL or head.next == NULL:
            return head
        repeat:
            set swapped = 0
            set ptr1 = head
            while ptr1.next != NULL:
                if ptr1.PRN > ptr1.next.PRN
                    swap(ptr1.PRN, ptr1.next.PRN)
                    swap(ptr1.Name, ptr1.next.Name)
                    swapped = 1
                ptr1 = ptr1.next
            until swapped == 0
            return head
End sortList
```

```
Algorithm mergelists (head1, head2):
        if head1 == NULL:
            return head2
        if head2 == NULL:
            return head1
        create dummy_node
        set tail = dummy_node
        while head1 != NULL and head2 != NULL:
            if head1.PRN <= head2.PRN:
                tail.next = head1
                head1 = head1.next
            else:
                tail.next = head2
                head2 = head2.next
            tail = tail.next
        if head1 != NULL :
            set tail.next = head1
        if head2 != NULL:
            set taid.next = head2
        return dummy-node.next
end mergelists
```

Complexities

Operation	Time Complexity	Auxilliary Space Com
Add President	O(1)	O(1)
Add secretary	O(N)	O(1)
Add Member	O(N)	O(1)
Delete President	O(1)	O(1)
Delete secretary	O(N)	O(1)
Delete Member	O(N)	O(1)
Display Members	O(N)	O(1)
Count Nodes	O(N)	O(1)
Reverse List	O(N)	O(1)
Sort List	$O(N^2)$	O(1)
Merge Two List	$O(N_1+N_2)$	O(1)

Conclusion:

In this practical, a Student Club enrollment management system was developed using a SLL. The prog. demonstrates dynamic memory allocation & efficient manipulation of linked ds. Operations such as insertion, deletion, traversal, counting, sorting, and reversing were successfully implemented.

FAQs:

1. Why is a LL preferred over a arr in this application? What info is stored in each node of the CLL?

Ans. A LL is preferred over an arr bcz it provides dynamic memory allocation & efficient O(1) insertion/deletion w/o requiring continuous pre allocated memory or element shifting, while each node of CLL stores data fields & alongside a pointer to the next node, with the final node linking back to the first node.

2. What is a SLL?

Ans. It is a linear ds consisting of dynamically allocated nodes chained together in a single direc^{n} where each node points to the next & last to NULL.

3. What info is stored in each node?

Ans. Each node in a SLL stores the actual value & a pointer to next memory loc.

4. What is dynamic memory allocation?

Ans. It is the process of allocating & reallocating memory space at runtime using pointers rather than fixing the size at compile time

5. Which func is used to release allocated memory?

Ans. The free() func is used to release dynamically allocated memory back to heap

6. What is the time complexity of inserting a node at the beginning?

Ans. The time complexity of inserting a node at the beginning of a SLL is O(1)

7. What is the time complexity of searching for a student?

Ans. The time complexity is O(N).

8. How is the linked list reversed?

Ans. A SLL is reversed iteratively using three pointers (prev, current, & next_node) to redirect each node's next pointer to its predecessor in a single pass.

9. Which sorting techniques can be used in a LL?

Ans. Merge Sort, Bubble Sort, & Insertion Sort.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    int PRN;
    char Name[30];
    struct Node *next;
};

struct Node *create_node(int prn, char name[]) {
    struct Node *new_node = (struct Node *)
malloc(sizeof(struct Node));
    new_node->PRN = prn;
    strcpy(new_node->Name, name);
    new_node->next = NULL;
    return new_node;
}

struct Node *addPresident(struct Node *head, int prn, char
name[]) {
    struct Node *new_node = create_node(prn, name);
    new_node->next = head;
    printf("President added successfully\n");
    return new_node;
}

struct Node *addSecretary(struct Node *head, int prn, char
name[]) {
```

```
    struct Node *new_node = create_node(prn, name);
    if (head == NULL) { printf("Secretary added
successfully\n"); return new_node; }
    struct Node *temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = new_node;
    printf("Secretary added successfully\n");
    return head;
}

struct Node *addMember(struct Node *head, int prn, char
name[], int pos) {
    struct Node *temp = head;
    for (int i = 1; i <= pos - 2; i++)
        if (temp != NULL) temp = temp->next;
    if (temp == NULL) { printf("Invalid position\n"); return head;
}
    struct Node *new_node = create_node(prn, name);
    new_node->next = temp->next;
    temp->next = new_node;
    printf("Member added successfully\n");
    return head;
}

struct Node *deletePresident(struct Node *head) {
    if (head == NULL) { printf("Club list is empty\n"); return
NULL; }
    struct Node *temp = head;
    head = head->next;
    free(temp);
```

```
    printf("President deleted successfully\n");
    return head;
}

struct Node *deleteSecretary(struct Node *head) {
    if (head == NULL) { printf("Club list is empty\n"); return
NULL; }
    if (head->next == NULL) { free(head); printf("Secretary
deleted successfully\n"); return NULL; }
    struct Node *temp = head;
    while (temp->next->next != NULL) temp = temp->next;
    free(temp->next);
    temp->next = NULL;
    printf("Secretary deleted successfully\n");
    return head;
}

struct Node *deleteMember(struct Node *head, int prn) {
    if (head == NULL) { printf("Club list is empty\n"); return
NULL; }
    if (head->PRN == prn) return deletePresident(head);
    struct Node *temp = head;
    while (temp->next != NULL && temp->next->PRN != prn)
temp = temp->next;
    if (temp->next == NULL) { printf("Member PRN not
found\n"); return head; }
    struct Node *del_node = temp->next;
    temp->next = del_node->next;
    free(del_node);
    printf("Member deleted successfully\n");
```

```
    return head;
}

void displayMembers(struct Node *head) {
    if (head == NULL) { printf("Club is empty\n"); return; }
    struct Node *temp = head;
    printf("Pinnacle Club Member List:\n");
    while (temp != NULL) {
        printf("PRN: %d, Name: %s\n", temp->PRN, temp->Name);
        temp = temp->next;
    }
}

int countMembers(struct Node *head) {
    int count = 0;
    struct Node *temp = head;
    while (temp != NULL) { count++; temp = temp->next; }
    return count;
}

struct Node *reverseList(struct Node *head) {
    struct Node *prev = NULL, *current = head, *next_node =
NULL;
    while (current != NULL) {
        next_node = current->next;
        current->next = prev;
        prev = current;
        current = next_node;
    }
    printf("List reversed successfully\n");
```

```
    return prev;
}

struct Node *sortList(struct Node *head) {
    if (head == NULL || head->next == NULL) return head;
    int swapped, t;
    char tname[30];
    struct Node *ptr1;
    do {
        swapped = 0;
        ptr1 = head;
        while (ptr1->next != NULL) {
            if (ptr1->PRN > ptr1->next->PRN) {
                t = ptr1->PRN; ptr1->PRN = ptr1->next->PRN; ptr1-
>next->PRN = t;
                strcpy(tname, ptr1->Name);
                strcpy(ptr1->Name, ptr1->next->Name);
                strcpy(ptr1->next->Name, tname);
                swapped = 1;
            }
            ptr1 = ptr1->next;
        }
    } while (swapped != 0);
    return head;
}

struct Node *mergeLists(struct Node *head1, struct Node
*head2) {
    if (head1 == NULL) return head2;
    if (head2 == NULL) return head1;
```

```
    struct Node dummy_node, *tail = &dummy_node;
    while (head1 != NULL && head2 != NULL) {
        if (head1->PRN <= head2->PRN) { tail->next = head1;
head1 = head1->next; }
        else { tail->next = head2; head2 = head2->next; }
        tail = tail->next;
    }
    if (head1 != NULL) tail->next = head1;
    if (head2 != NULL) tail->next = head2;
    return dummy_node.next;
}

int main() {
    struct Node *head = NULL, *head2 = NULL;
    int ch, prn, pos, n;
    char name[30];
    do {
        printf("\n1.Add President\n" "2.Add Secretary\n" "3.Add
Member\n" "4.Delete President\n"
               "5.Delete Secretary\n" "6.Delete Member\n"
"7.Display\n" "8.Count\n" "9.Sort\n"
               "10.Reverse\n" "11.Merge with second list \n"
"12.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("PRN Name: "); scanf("%d %s", &prn,
name);
                    head = addPresident(head, prn, name); break;
            case 2: printf("PRN Name: "); scanf("%d %s", &prn,
name);
```

```
            head = addSecretary(head, prn, name); break;
        case 3: printf("PRN Name Position: "); scanf("%d %s
%d", &prn, name, &pos);
            head = addMember(head, prn, name, pos); break;
        case 4: head = deletePresident(head); break;
        case 5: head = deleteSecretary(head); break;
        case 6: printf("PRN to delete: "); scanf("%d", &prn);
            head = deleteMember(head, prn); break;
        case 7: displayMembers(head); break;
        case 8: printf("Total members: %d\n",
countMembers(head)); break;
        case 9: head = sortList(head); displayMembers(head);
break;
        case 10: head = reverseList(head);
displayMembers(head); break;
        case 11: printf("How many members in second list: ");
scanf("%d", &n);
            for (int i = 0; i < n; i++) {
                printf("PRN Name: "); scanf("%d %s", &prn,
name);
                head2 = addSecretary(head2, prn, name);
            }
            head = mergeLists(sortList(head),
sortList(head2));
            head2 = NULL;
            displayMembers(head); break;
        case 12: printf("Exiting\n"); break;
        default: printf("Invalid choice\n");
    }
  } while (ch != 12);
```

```
    return 0;
}
```

Input and Output:

TC1:

1

101 Riya

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:1

PRN Name:101

Riya

President added successfully
```

TC2:

2

199 Aman

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:2

PRN Name:199 Aman

Secretary added successfully
```

TC3:

3

150 Neha 2

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:3

PRN Name Position:150 Neha 2

Member added successfully
```

TC4:

3

120 Karan 3

3

170 Sneha 4

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:3

PRN Name Position:120 Karan 3

Member added successfully
```

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:3

PRN Name Position:170 Sneha 4

Member added successfully
```

TC5:

7

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:7

Club is empty
```

TC6:

7

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:7

Pinnacle Club Member List:
PRN: 101, Name: Riya
PRN: 150, Name: Neha
PRN: 120, Name: Karan
PRN: 170, Name: Sneha
PRN: 199, Name: Aman
```

TC7:

6

150

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:6

PRN to delete:150

Member deleted successfully
```

TC8:

4

5

7

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:4

President deleted successfully

1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:5

Secretary deleted successfully

1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:7

Pinnacle Club Member List:
PRN: 120, Name: Karan
PRN: 170, Name: Sneha
```

TC9:

6

999

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:6

PRN to delete:999

Member PRN not found
```

TC10:

8

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:8

Total members: 5
```

TC11:

9

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:9

Pinnacle Club Member List:
PRN: 101, Name: Riya
PRN: 120, Name: Karan
PRN: 150, Name: Neha
PRN: 170, Name: Sneha
PRN: 199, Name: Aman
```

TC12:

11

2

130 Om

110 Isha

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:11

How many members in second list:2

PRN Name:130 Om

Secretary added successfully
PRN Name:110 Isha

Secretary added successfully
Pinnacle Club Member List:
PRN: 110, Name: Isha
PRN: 120, Name: Karan
PRN: 130, Name: Om
PRN: 170, Name: Sneha
```

TC13:

10

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:10

List reversed successfully
Pinnacle Club Member List:
PRN: 170, Name: Sneha
PRN: 120, Name: Karan
```

TC14:

4

4

4

4

7

```
1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:4

President deleted successfully

1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
```

```
11.Merge with second list
12.Exit
Choice:4

President deleted successfully

1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:4

President deleted successfully

1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
```

```
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:4

President deleted successfully

1.Add President
2.Add Secretary
3.Add Member
4.Delete President
5.Delete Secretary
6.Delete Member
7.Display
8.Count
9.Sort
10.Reverse
11.Merge with second list
12.Exit
Choice:7

Club is empty
```